Experiment - 1

Aim:-

To study and implement advanced SQL queries using conditional logic, aggregate functions, joins, subqueries, self-joins, and date operations to solve real-world database problems

Question 1)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The salary categories are:

- Low Salary: Monthly salary less than 20000
- Average Salary: Monthly salary between 20000 and 50000 (inclusive)
- High Salary: Monthly salary greater than 50000

The final output should always contain all three salary categories even if one category contains zero accounts.

Answer

```
select 'Low Salary' as Category,
count(account_id) as account_count from accounts
where income<20000
union all
select 'Average Salary' as Category,
count(account_id) as account_count from accounts
where income>=20000 and income<=50000
union all
select 'High Salary' as Category,
count(account_id) as account_count from accounts
where income>50000
```

Output

	category text	account_count bigint
1	Low Salary	1
2	Average Sala...	1
3	High Salary	3

Question 2)

Design an Employee Management System database architecture by performing the following operations:

• Create Departments, Employees and Salaries tables.

• Insert sample records.

• Display employee details using JOIN.

• Increase HR employee salary by 10%.

• Delete salary records below 30000.

• Display employees earning more than the company average salary.

Answer

1) Display Employee Details

```
select e.name, d.dept_name from employees as e
inner join departments as d
on e.dept_id=d.dept_id
inner join salaries as s
on e.emp_id=s.emp_id
```

2) Increase HR Salary by 10%

```
update salaries
set salary = salary + salary*0.10
where emp_id in (
select e.emp_id from employees as e
join departments as d
on e.dept_id=d.dept_id
where d.dept_name='HR'
)
```

3) Delete Salaries below 30000

```
DELETE FROM Salaries
WHERE salary <30000;
```

4) Display above average salaries

```
select * from employees as e
join salaries as s
on e.emp_id=s.emp_id
where s.salary>(select avg(salary) from salaries)
```

Output

1) Employee Details

	name character varying (50)	dept_name character varying (50)
1	Alice	HR
2	Bob	HR
3	Charlie	IT
4	David	IT
5	Emma	Sales

2) Increased Salary

```
UPDATE 2
```

3) Delete salaries below 30000

```
DELETE 1
```

4) Above average salaries

	emp_id integer	name character varying (50)	dept_id integer	emp_id integer	salary integer
1	3	Charlie	20	3	80000

Question 3)

Generate every possible pizza consisting of exactly three different toppings. Display the topping names separated by commas along with the total ingredient cost. Sort the result by highest cost and alphabetically for ties.

Answer

```
SELECT
    (a.topping_name,
    b.topping_name,
    c.topping_name) as pizza,
    a.ingredient_cost + b.ingredient_cost + c.ingredient_cost as total_cost
FROM pizza_toppings a, pizza_toppings b, pizza_toppings c
WHERE a.topping_name < b.topping_name
  AND b.topping_name < c.topping_name
order by total_cost desc, pizza asc;
```

Output

	pizza record	total_cost numeric
1	Chicken,Pepperoni,Sausage	1.75
2	Chicken,Extra Cheese,Sausage	1.65
3	Extra Cheese,Pepperoni,Saus...	1.60
4	Chicken,Onions,Sausage	1.50
5	Chicken,Extra Cheese,Pepper...	1.45
6	Onions,Pepperoni,Sausage	1.45
7	Extra Cheese,Onions,Sausage	1.35
8	Chicken,Onions,Pepperoni	1.30
9	Chicken,Extra Cheese,Onions	1.20
10	Extra Cheese,Onions,Pepperoni	1.15

Question 4)

Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

Requirements

1. Compare purchases **made by the same user only**.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur **after** the first purchase.

4. Same-day purchases must **not** be considered.
5. The second purchase must occur within **7 days** of the first purchase.
6. Display only the **unique User IDs** that satisfy the above conditions.

Sort the output in ascending order of user_id.

Answer

```sql
SELECT DISTINCT a.user_id
FROM amazon_transactions a,
     amazon_transactions b
WHERE a.user_id = b.user_id
  AND b.created_at > a.created_at
  AND b.created_at - a.created_at >= INTERVAL '1 day'
  AND b.created_at - a.created_at <= INTERVAL '7 days'
ORDER BY a.user_id;
```

Output

	user_id integer
1	101
2	103